



Cyber Security

WEEK 2

Charlotte Emma Whiting

CONTENT - WEEK 2

27-34

Q1. E2EE IN VOTING SYSTEMS

35-39

Q2. PRIMARY MODES IN SNORT

40-41

Q3. PRACTICAL / ICE ROOM AND METASPLOIT

42 - 45

Q4. PRACTICAL / SECURING LINUX SYSTEM

Q1.

END TO END

ENCRYPTION IN

VOTING SYSTEMS



END TO END ENCRYPTION



Definition

End-to-end encryption (E2EE) is a method of secure communication that prevents third parties from accessing data while it's transferred from one end system or device to another.

In E2EE, the data is encrypted on the sender's system or device, and only the intended recipient can decrypt it. As it travels to its destination, the message cannot be read or tampered with by an internet service provider, application service provider, hacker or any other entity or service.

The cryptographic keys used to encrypt and decrypt the messages are stored on the endpoints.

ENCRYPTION OF VOTES

It is important to maintain integrity and secrecy during the voting process so E2EE during both transmission and storage is paramount. There is multiple stages to ensure that legislations are followed.

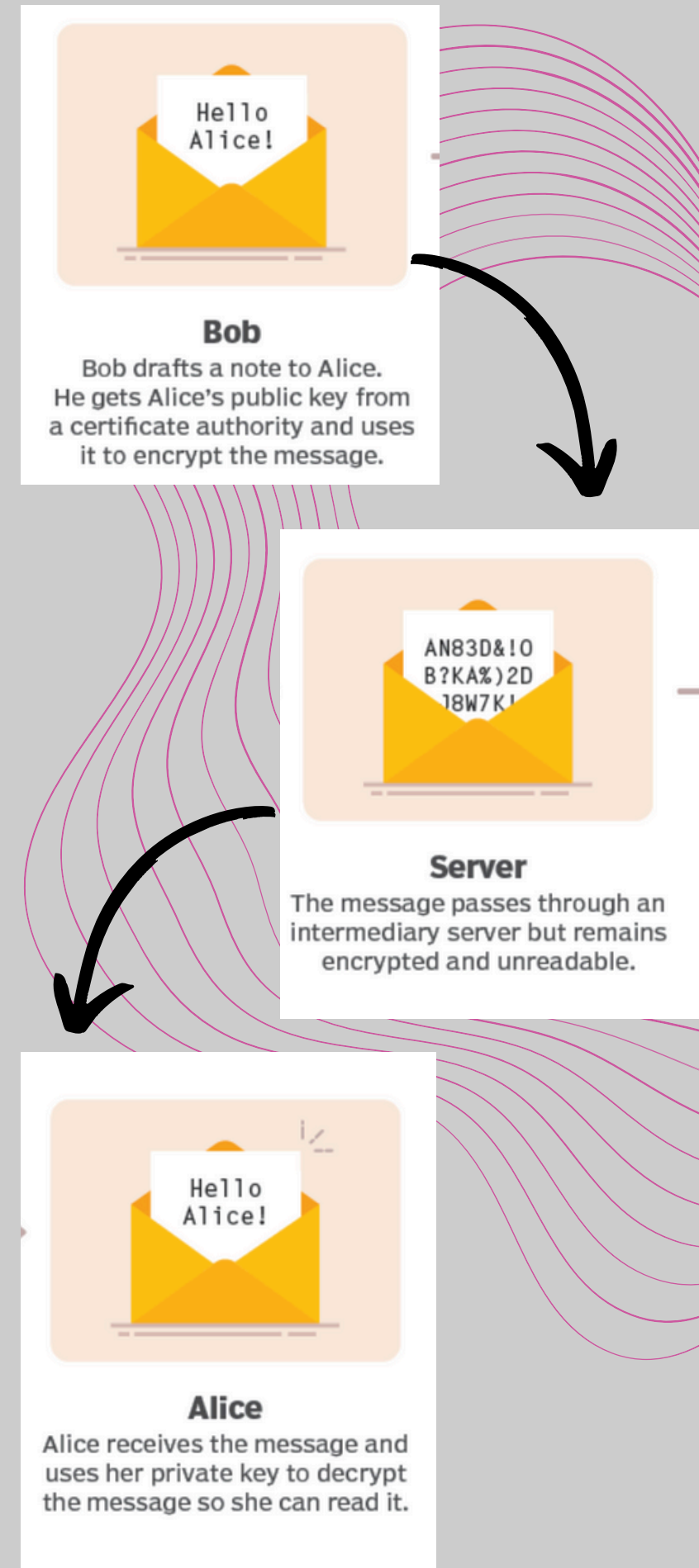
ENCRYPTION DURING VOTING

- We would need to make sure that when a voter casts their ballot, the vote is immediately encrypted on the voter's device using a public key provided by the election authority. This public key will only able to encrypt data. Each vote can be signed by the voter's private key (or through an authentication method) to ensure the vote's authenticity and prevent tampering or double voting.

SEND THE ENCRYPTED DATA TO THE ELECTION SERVER

- After the vote has been encrypted it needs to be sent via the internet to the election server. The votes must be encrypted to ensure that anyone intercepting the communication cannot read or alter the vote, maintaining both confidentiality and integrity during this process.

Since the encryption is end-to-end, not even the server processing the vote can decrypt the vote only the designated authority holding the private key will be able to decrypt the data. This ensures that the voting system is tamper-proof.



SECURE STORING OF VOTES

Now we have the encrypted data, we must ensure that it is stored securely to maintain the integrity of the voting system.

SECURE STORAGE

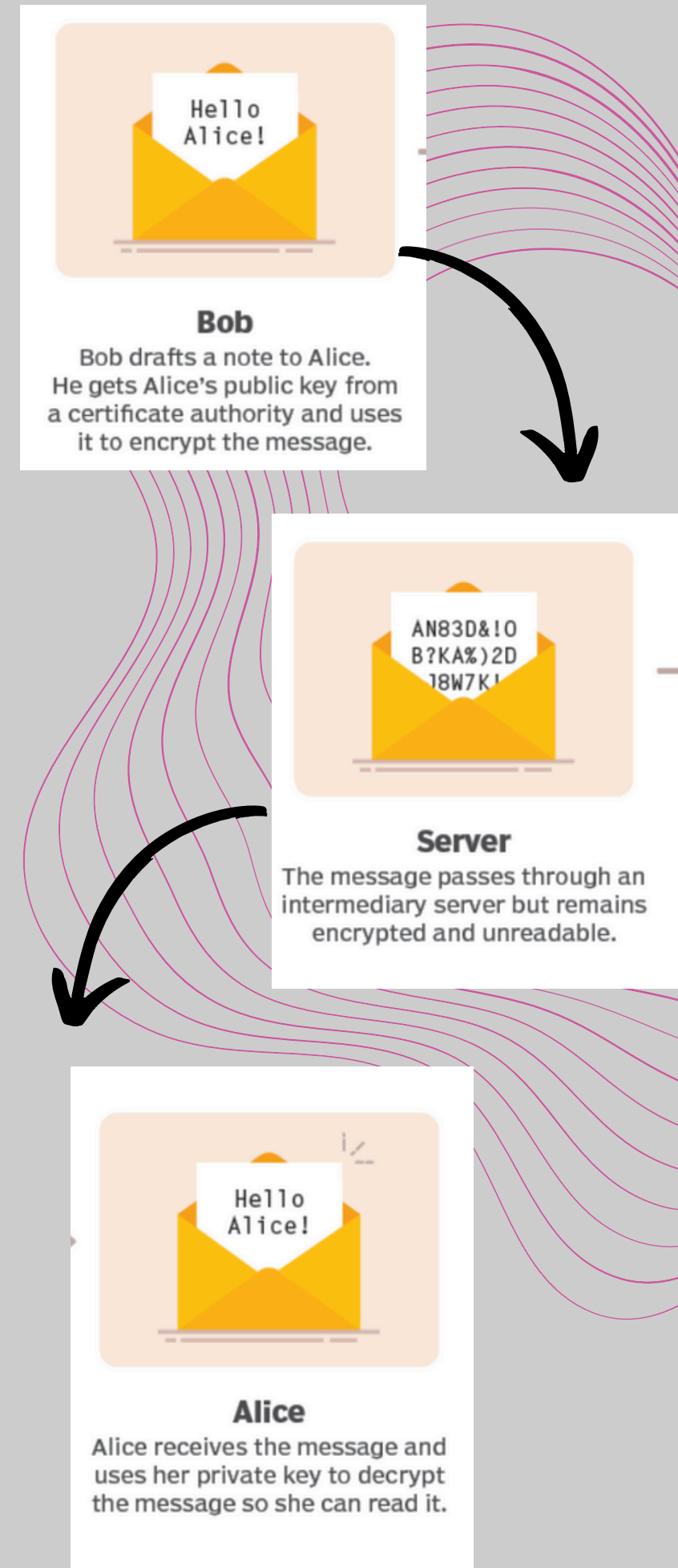
- Once the votes reach the election servers they will remain encrypted. The server will be unable to decrypt them preventing unauthorised access to the votes while they are stored. This ensures that the votes will be safe even if the server is compromised.

BLOCKCHAIN STORAGE / DISTRIBUTED LEDGER

- Encrypted votes can also be stored on a distributed ledger like a blockchain to further enhance the integrity and auditability of the voting process.

Definition of blockchain

Blockchain is a distributed ledger with growing lists of records(blocks) that are securely linked together via cryptographic hashes. Each block contains a cryptographic hash of the previous block, timestamp and transaction data. Since each block contains information about the previous block, they effectively form a chain (compare linked list data structure), with each additional block linking to the ones before it. Consequently, blockchain transactions are irreversible in that, once they are recorded, the data in any given block cannot be altered retroactively without altering all subsequent blocks.



DECRYPTION

PRIVATE KEY DECRYPTION

- Once all votes are in, the authority with the corresponding private key will decrypt the votes. This will be a unique key that only the designated election authority can access so they can tally the votes.

VOTE TALLYING

- The decryption will be completed to ensure the vote remains anonymous and the count accurate. Techniques like homomorphic encryption or zero-knowledge proofs can be used to tally votes without revealing individual votes.

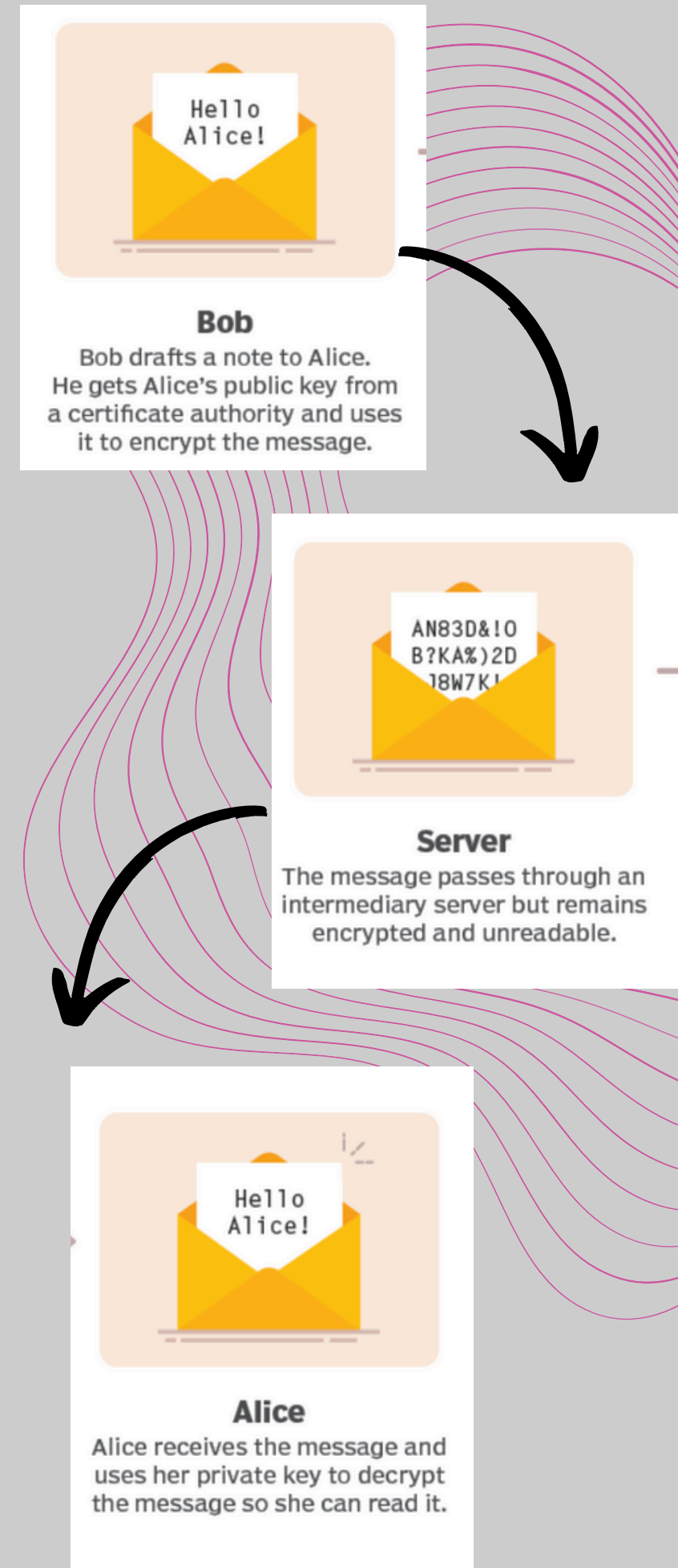
VERIFICATION

VOTING VERIFICATION

- After casting their encrypted vote, the system should provide the voter with a receipt to verify that their vote has been received. This verification step is important for trust but must be implemented in a way that doesn't compromise vote secrecy.

AUDITS

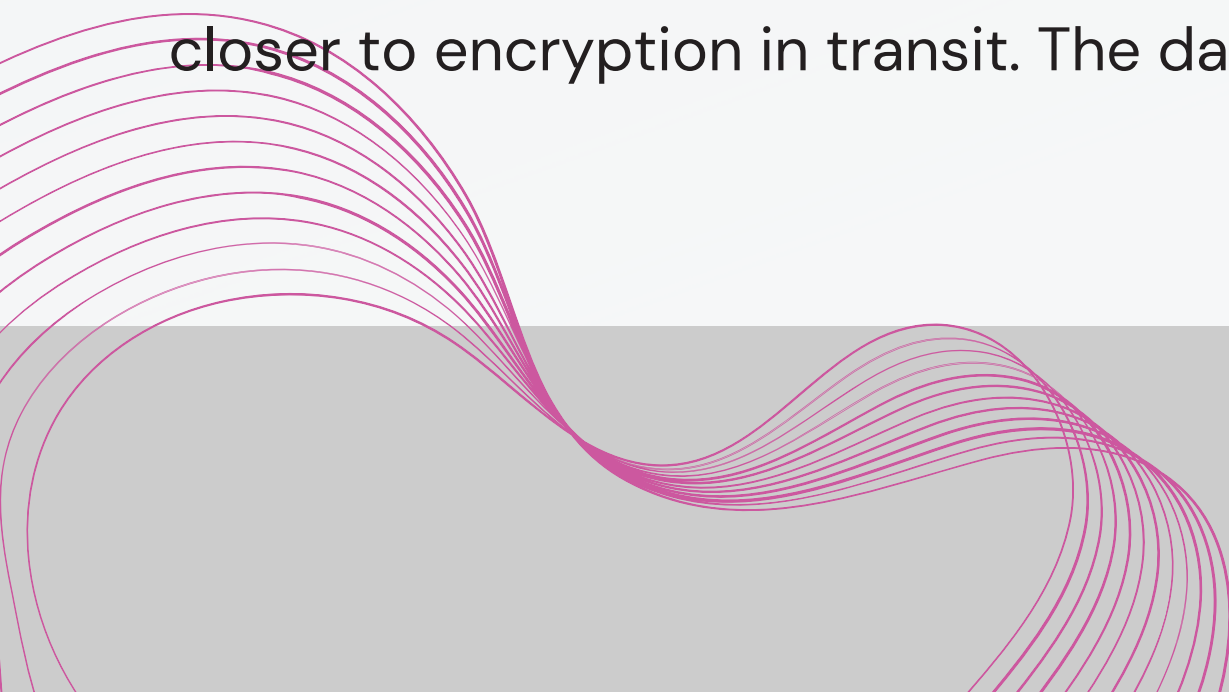
- Cryptographic techniques like end-to-end verifiability will allow independent auditors to verify that the election results are consistent and true, without revealing the content of any individual vote.



CHALLENGES AND CONSIDERATIONS

End to end encryption key exchange is considered unbreakable using known algorithms and current computing power, however there are identified potential weaknesses of the encryption scheme. It can not be protected against things such as:

- **Metadata.** While E2EE protects the information inside a message, it does not conceal information about the message, such as the date and time it was sent or the participants in the exchange. This metadata could give malicious actors with an interest in the encrypted information clues as to where they may be able to intercept the information once it has been decrypted.
- **Compromised endpoints.** If either endpoint has been compromised, an attacker may be able to see a message before it is encrypted or after it is decrypted. Attackers could also retrieve keys from compromised endpoints and execute a man-in-the-middle attack with a stolen public key.
- **Vulnerable intermediaries.** Sometimes, providers claim to offer end-to-end encryption when what they really offer is closer to encryption in transit. The data may be stored on an intermediary server where it can be accessed.



DISADVANTAGES

End to end encryption generally does a good job of securing digital communications, however it does not guarantee data security. There are some disadvantages to consider:

- **Complexity in defining the endpoints.** Some E2EE implementations allow the encrypted data to be decrypted and reencrypted at certain points during transmission. So it is important to minimise these for an election and to make sure that the endpoints are clearly defined and distinguished along the communication circuit.
- **Visible metadata.** The vote messages in transit are encrypted and impossible to read, however there is information about the message such as the date sent and recipient is still visible, which may provide useful information to an unauthorised person.
- **Endpoint security.** If endpoints are compromised, encrypted data may be revealed.
- **Not future-proof.** Although end-to-end encryption is a strong technology now, there is speculation that eventually quantum computing will render cryptography obsolete.



Q2. PRIMARY MODES FOR SNORT



NIDS



NETWORK INTRUSION DETECTION SYSTEM (NIDS)

In NIDS Mode, Snort will act as a full-fledged intrusion detection system. It monitors network traffic in real-time and compares it against a set of predefined rules to detect potential threats, such as port scans, suspicious activities, or known attack signatures. Snort will then generate alerts when it detects malicious behaviour. This mode is Snort's main mode of operation.

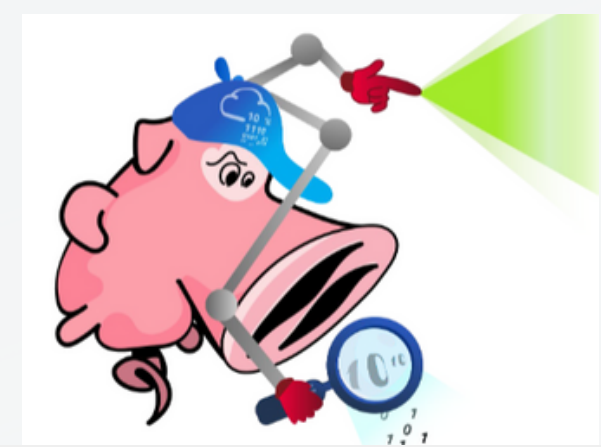
In this mode Snort will:

- Detect network scans, malware infections, SQL injection threats, and other intrusions.
- In addition to detecting threats, you can configure Snort to respond by blocking traffic from a certain IP address or port.

Scenerio

An organisation wants to secure its internal network from external threats like port scans, SQL injections, or denial-of-service attacks. By configuring Snort in NIDS mode with appropriate rule sets, the system will monitor the network traffic and alert the security team if it detects any anomalies or known attack signatures

SNIFFER



SNIFFER MODE

Sniffer mode is actively sniffing out threats, It's a passive listening mode, always looking at network traffic without taking any alert or blocking an action.

It doesn't interfere with network traffic, but when it if it detects a match between a rule you've set and a suspicious bit of traffic, it will generate an alert or log. It is not used for prevention, but for monitoring.

In this mode, Snort will

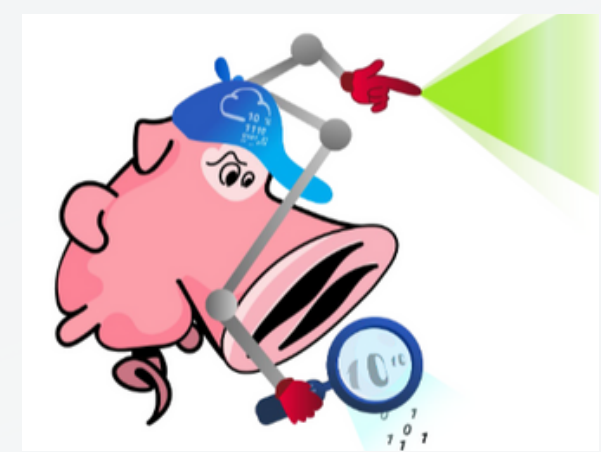
- **Captures and logs network traffic:** It does this by simulating a packet sniffer.
- **If configured:** Write the intercepted packets to a file, display them on the console, or upload them to a remote server for further analysis.

Scenerio

A network administrator needs to run some troubleshooting on network performance. They want to observe the raw traffic to see if there are any abnormalities, such as high latency or unusual protocols being used. By running Snort in Sniffer Mode, the administrator can view real-time traffic and analyze it for immediate insights.

SNORT MODES

SNIFFER MODE – SUDO SNORT -D

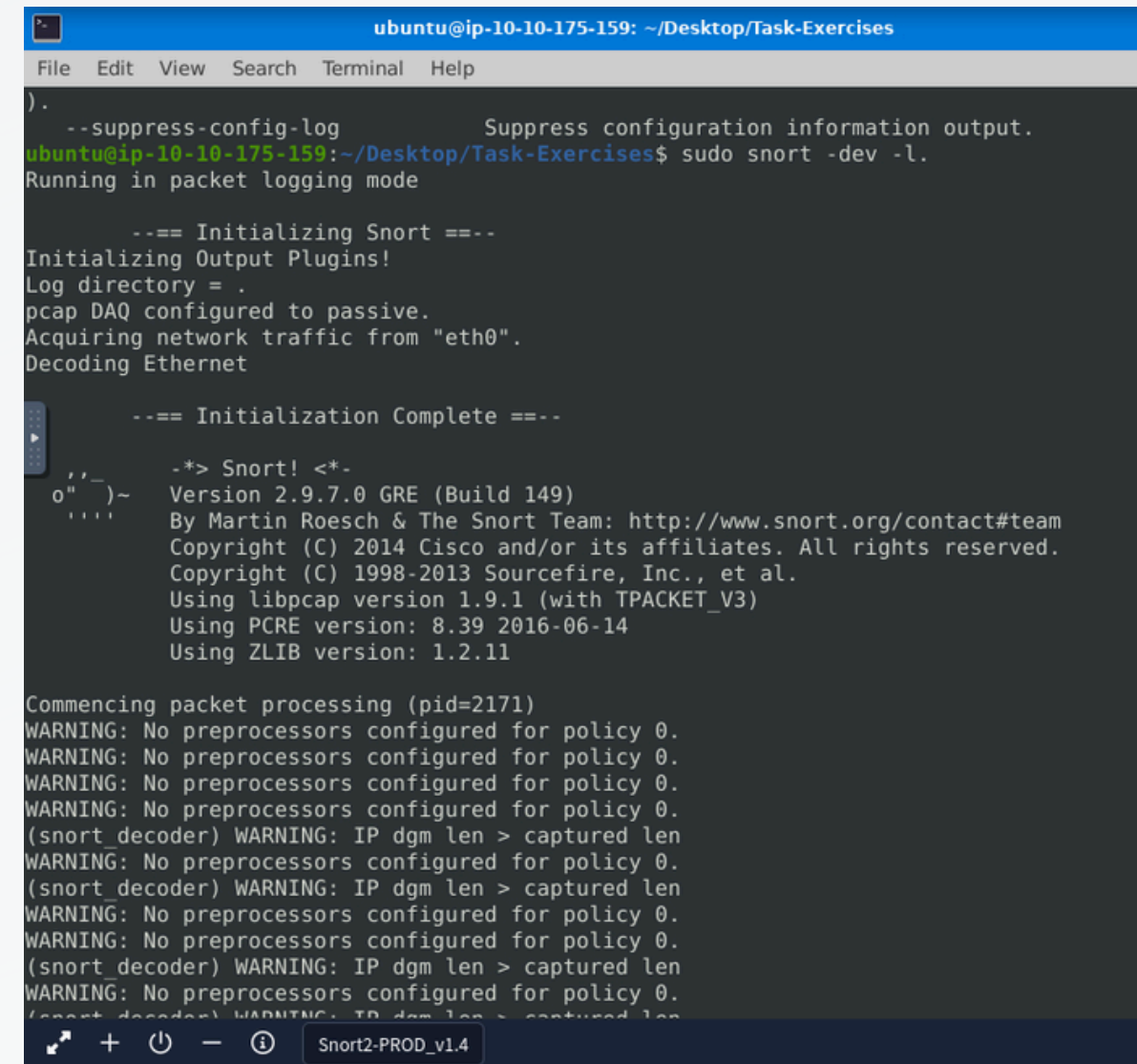


```
ubuntu@ip-10-10-175-159: ~/Desktop/Task-Exercises
File Edit View Search Terminal Help
^C*** Caught Int-Signal
WARNING: No preprocessors configured for policy 0.
=====
Run time for packet processing was 173.855771 seconds
Snort processed 19983 packets.
Snort ran for 0 days 0 hours 2 minutes 53 seconds
  Pkts/min:      9991
  Pkts/sec:       115
=====
Memory usage summary:
  Total non-mmapped bytes (arena):      786432
  Bytes in mapped regions (hblkhd):     13213696
  Total allocated space (uordblks):      679312
  Total free space (fordblks):           107120
  Topmost releasable block (keepcost):   104480
=====
Packet I/O Totals:
  Received:      20034
  Analyzed:      19983 ( 99.745%)
  Dropped:        0 (  0.000%)
  Filtered:       0 (  0.000%)
  Outstanding:   51 (  0.255%)
  Injected:       0
=====
Breakdown by protocol (includes rebuilt packets):
  Eth:           19983 (100.000%)
  VLAN:          0 (  0.000%)
  IP4:           19977 ( 99.970%)
  Frag:          0 (  0.000%)
  ICMP:          0 (  0.000%)
  UDP:           0 (  0.000%)
  TCP:          14964 ( 74.884%)
  IP6:           0 (  0.000%)
  IP6 Ext:       0 (  0.000%)
  IP6 Opts:      0 (  0.000%)
  Frag6:         0 (  0.000%)
=====
Snort2-PROD_v1.4 16min 4s
```


PACKET LOGGER

PACKET LOGGER

In Packet Logger Mode, Snort is configured to capture and log all network packets to a specified directory on the disk. It captures the packets and stores them for later analysis, providing a more permanent record compared to Sniffer Mode. This mode is used for network analysis, troubleshooting and security monitoring

A screenshot of a terminal window titled 'ubuntu@ip-10-10-175-159: ~/Desktop/Task-Exercises'. The terminal shows the command 'sudo snort -dev -l.' being executed. The output indicates that Snort is running in packet logging mode, initializing output plugins, and capturing network traffic from 'eth0'. It also displays version information and copyright notices for Snort 2.9.7.0 GRE (Build 149).

```
File Edit View Search Terminal Help
).
--suppress-config-log          Suppress configuration information output.
ubuntu@ip-10-10-175-159:~/Desktop/Task-Exercises$ sudo snort -dev -l.
Running in packet logging mode

--== Initializing Snort ==--
Initializing Output Plugins!
Log directory = .
pcap DAQ configured to passive.
Acquiring network traffic from "eth0".
Decoding Ethernet

--== Initialization Complete ==--

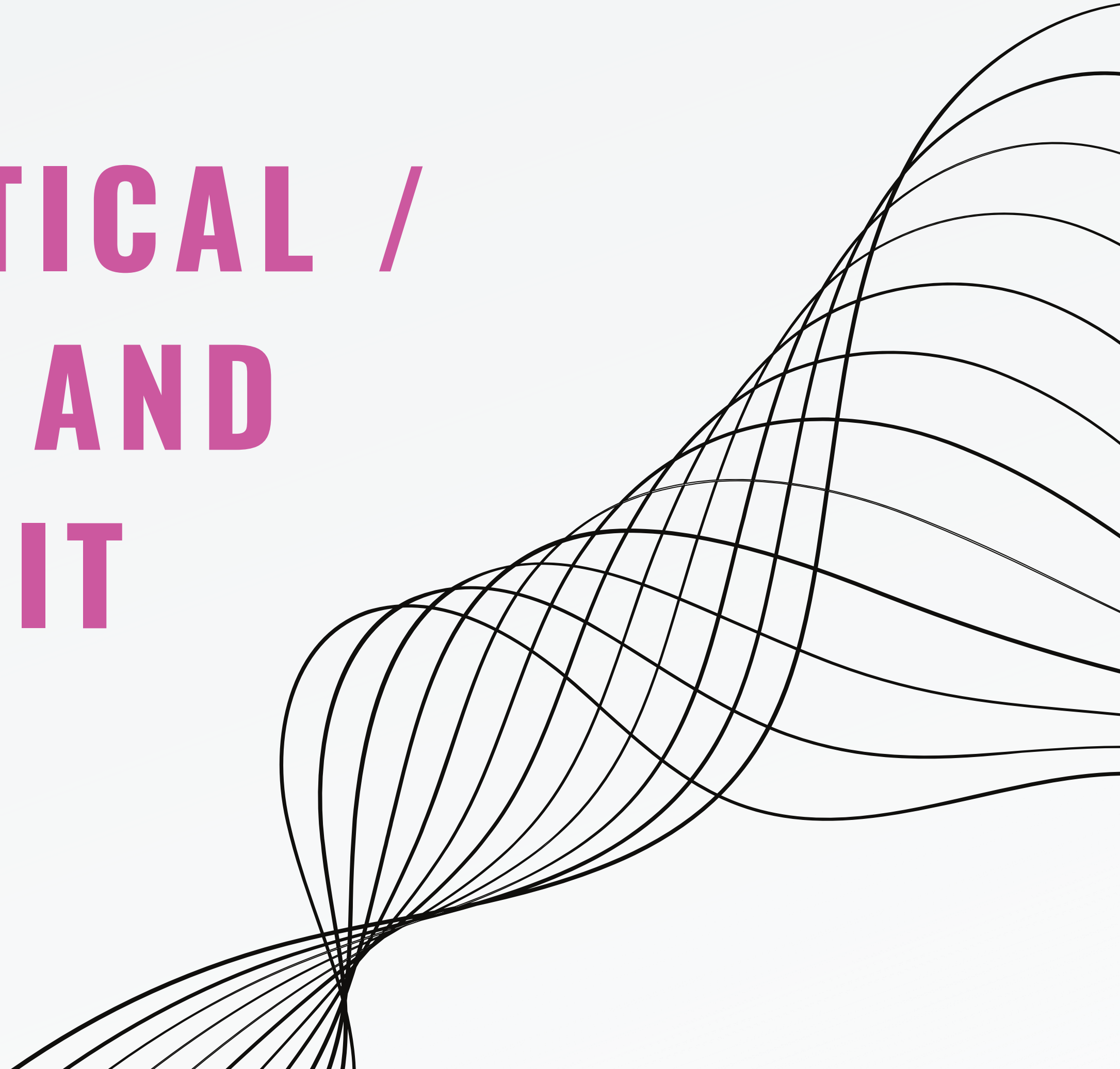
--> Snort! <*-
Version 2.9.7.0 GRE (Build 149)
By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
Copyright (C) 2014 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.9.1 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Commencing packet processing (pid=2171)
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
(snort_decoder) WARNING: IP dgm len > captured len
WARNING: No preprocessors configured for policy 0.
(snort_decoder) WARNING: IP dgm len > captured len
WARNING: No preprocessors configured for policy 0.
WARNING: No preprocessors configured for policy 0.
(snort_decoder) WARNING: IP dgm len > captured len
WARNING: No preprocessors configured for policy 0.
(snort_decoder) WARNING: IP dgm len > captured len
```

Scenerio

A company wants to retain a history of network traffic over a certain timeframe to investigate a potential insider threat. The security team can run Snort in Packet Logger Mode, saving all packets to a file for detailed analysis later on and to potentially use as evidence to support the investigation.

Q3. PRACTICAL / ICE ROOM AND METASPLOIT



TRY HACK ME - ICE ROOM

What is the CVE number for this vulnerability?

Metasploit modules for CVE-2004-1561

 **Icecast Header Overwrite** Disclosure Date: 2004-09-28 First seen: 2020-04-26

exploit/windows/http/icecast_header

This module exploits a buffer overflow in the header parsing of icecast versions 2.0.1 and earlier, discovered by Luigi Auriemma. Sending 32 HTTP headers will cause a write one past the end of a pointer array. On win32 this happens to overwrite the saved instruction

[More information](#)

CVSS scores for CVE-2004-1561

Base Score	Base Severity	CVSS Vector	Exploitability Score	Impact Score	Score Source	First Seen
7.5	HIGH	AV:N/AC:L/Au:N/C:P/I:P/A:P	10.0	6.4	NIST	

What is the command to search for the vulnerability related to the CVE in msfconsole?

```
msf6 > search cve:CVE-2004-1561
```

Q4.

SECURING LINUX

SERVER



SECURING A LINUX SERVER HOSTING AN E-COMMERCE WEB APPLICATION



Task

Your responsible for managing a Linux server that hosts an e-commerce web application. The primary goals are to enhance the server's security and protect it from unauthorised access.

Things to consider are:

- The server uses NGINX to serve the e-commerce site
- It has HTTPS enabled through Let's Encrypt SSL certificates.
- The web application consists of a React front-end and a Node.js API running on port 3001.
- There is data stored in a MySQL database running on its default port.

Your task is to establish a firewall using UFW (Uncomplicated Firewall) to safeguard the server.

STEP BY STEP

To start, we would need to configure the firewall rules. The setup must include protecting the server while allowing necessary access to the application, API, and SSH, and blocking remote access to the MySQL database to ensure security for the data.

1. INSTALL THE UNCOMPLICATED FIREWALL (UFW)

```
(charlotteemmawhiting@kali)-[~]  
$ sudo ufw enable  
Firewall is active and enabled on system startup
```

2. ENABLE THE UFW

3. ALLOW ACCESS TO NGINX (BOTH HTTP AND HTTPS TRAFFIC)

This will allow both HTTP (port 80) and HTTPS (port 443) traffic.

```
(charlotteemmawhiting@kali)-[~]  
$ sudo ufw allow 'Nginx Full'  
Rule added  
Rule added (v6)
```

```
charlotteemmawhiting@kali: ~  
File Actions Edit View Help  
  
(charlotteemmawhiting@kali)-[~]  
$ sudo apt update  
[sudo] password for charlotteemmawhiting:  
Get:1 http://ftp.free.fr/pub/kali kali-rolling InRelease [41.5 kB]  
Get:2 http://ftp.free.fr/pub/kali kali-rolling/main amd64 Packages  
Get:3 http://ftp.free.fr/pub/kali kali-rolling/main amd64 Contents  
2 MB]  
Get:4 http://ftp.free.fr/pub/kali kali-rolling/contrib amd64 Packag  
]  
Get:5 http://ftp.free.fr/pub/kali kali-rolling/contrib amd64 Cont  
269 kB]  
Get:6 http://ftp.free.fr/pub/kali kali-rolling/non-free amd64 Packa  
B]  
Get:7 http://ftp.free.fr/pub/kali kali-rolling/non-free amd64 Conte  
[873 kB]  
Fetched 70.8 MB in 17s (4136 kB/s)  
1298 packages can be upgraded. Run 'apt list --upgradable' to see t  
  
(charlotteemmawhiting@kali)-[~]  
$ sudo apt install ufw  
Installing:  
  ufw  
  
Summary:  
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 1298  
  Download size: 168 kB  
  Space needed: 880 kB / 4319 MB available  
  
Get:1 http://kali.download/kali kali-rolling/main amd64 ufw all 0.3  
kB]  
Fetched 168 kB in 3s (53.0 kB/s)  
Preconfiguring packages ...  
Selecting previously unselected package ufw.  
(Reading database ... 391453 files and directories currently instal  
Preparing to unpack .../archives/ufw_0.36.2-6_all.deb ...  
Unpacking ufw (0.36.2-6) ...  
Setting up ufw (0.36.2-6) ...  
  
Creating config file /etc/ufw/before.rules with new version  
  
Creating config file /etc/ufw/before6.rules with new version  
  
Creating config file /etc/ufw/after.rules with new version  
  
Creating config file /etc/ufw/after6.rules with new version  
update-rc.d: We have no instructions for the ufw init script.  
update-rc.d: It looks like a non-network service, we enable it.
```

4. ALLOW ACCESS TO NODE.JS API (PORT 3001)

```
(charlotteemmawhiting@kali)-[~]  
$ sudo ufw allow 3001/tcp  
Rule added  
Rule added (v6)
```

5. BLOCK REMOTE ACCESS TO MYSQL (PORT 3306)

```
(charlotteemmawhiting@kali)-[~]  
$ sudo ufw deny 3306/tcp  
Rule added  
Rule added (v6)
```

6. ALLOW SSH ACCESS (PORT 22)

```
(charlotteemmawhiting@kali)-[~]  
$ sudo ufw allow 22/tcp  
Rule added  
Rule added (v6)
```

CHECK UFW STATUS

Once complete, test the following:

- The web application (React front-end) should be accessible on HTTP/HTTPS.
- The Node.js API should be accessible on port 3001.
- MySQL should not be remotely accessible.
- SSH should work, allowing you to manage the server remotely.

```
(charlotteemmawhiting@kali)-[~]  
$ sudo ufw status  
Status: active
```

To	Action	From
--	---	---
Nginx Full	ALLOW	Anywhere
3001/tcp	ALLOW	Anywhere
3306/tcp	DENY	Anywhere
22/tcp	ALLOW	Anywhere
Nginx Full (v6)	ALLOW	Anywhere (v6)
3001/tcp (v6)	ALLOW	Anywhere (v6)
3306/tcp (v6)	DENY	Anywhere (v6)
22/tcp (v6)	ALLOW	Anywhere (v6)